H2 SF2525

Tommaso Brambilla

brambi@kth.se

1 Exercise 1

Given the stochastic differential equation

$$dX(t) = a(X(t), t)dt + b(X(t), t)dW(t)$$

with initial condition $X(0) = x_0$, we want to write a computer program that computes the forward Euler approximation $\bar{X}$ and test the convergence rate of both the strong error

$$\|X(T) - \bar{X}(T)\|_{L^2} = \sqrt{\mathbb{E}[(X(T) - \bar{X}(T))^2]}$$

and the weak error

$$\mathbb{E}[g(X(T)) - g(\bar{X}(T))]$$

with functions a , b and g that satisfy conditions stated in theorems 3.1 and 5.8.
When performing numerical tests on an SDE with a known exact solution, I will consider the SDE

$$dS = \mu S dt + \sigma S dW_t$$

and its analytical solution

$$S(t) = S(0)e^{(\mu - \frac{\sigma^2}{2})t - \sigma W(t)}$$

At first I used a function $g(x) = x$, which is differentiable to any order and its derivatives are bounded, so that g satisfies theorem 5.8. Running the code, I obtained that the weak error converged with a linear rate with respect to the time step dt, while the strong error convergence following $\sqrt{dt}$

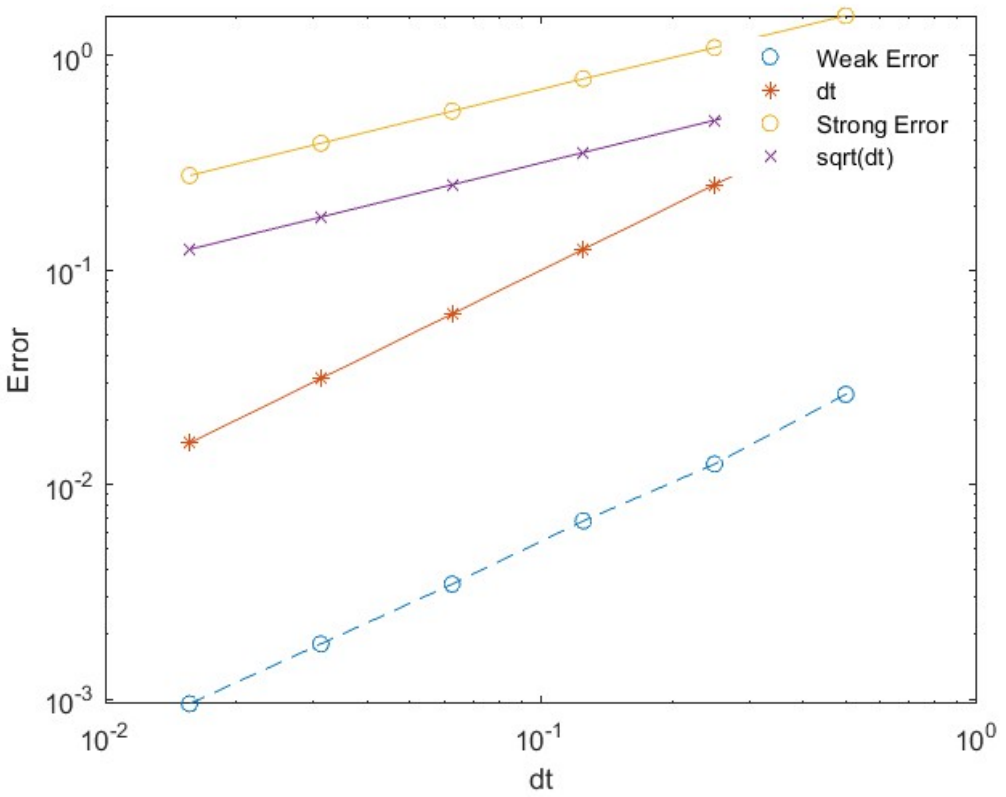


Figure 1: g(x)=x

Then I changed function g to $g(x) = e^x$ in order to check the convergence rate of the weak error using a function whose derivatives are not bounded and, as we can see in Figure 2, the weak error does not seem to converge anymore.

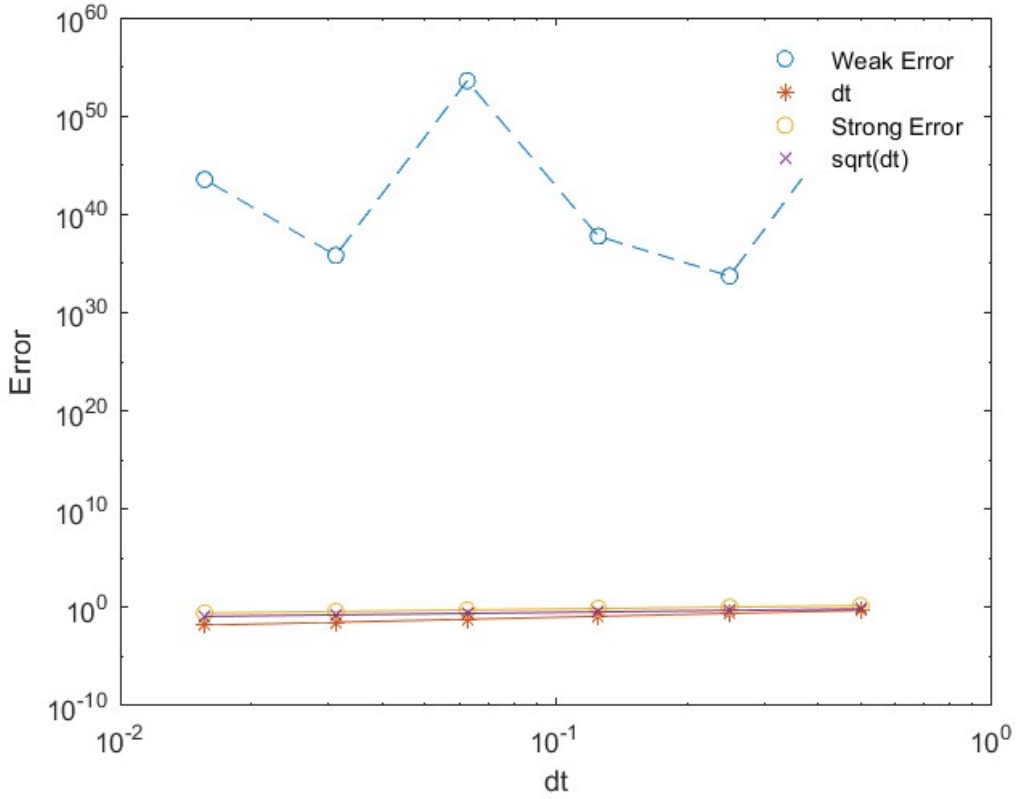
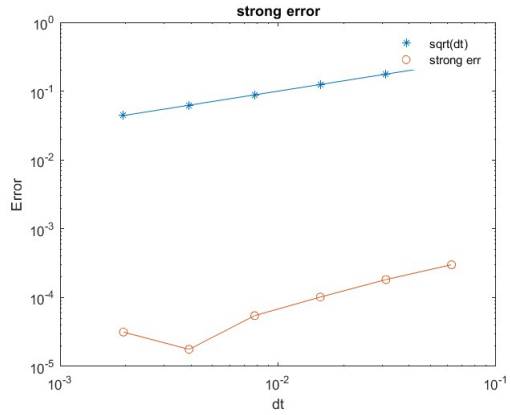
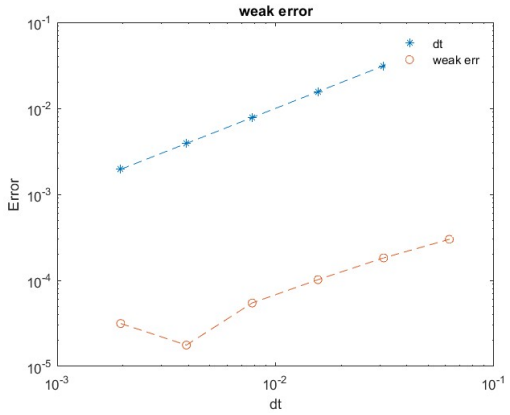


Figure 2: g(x)=e^x

Then I tried to implement a code to compute the convergence rate of strong and weak error of forward Euler approximation of an SDE without a known exact solution. The main idea that I tried to apply is to build a "reference solution" that could use as exact solution, and I built it using a much finer refinement of forward Euler than the one I used for the forward Euler approximations used as numerical tests. Below there are the results obtained when using $g(x) = x$ (Figure 3) and $g(x) = e^x$ (Figure 4).

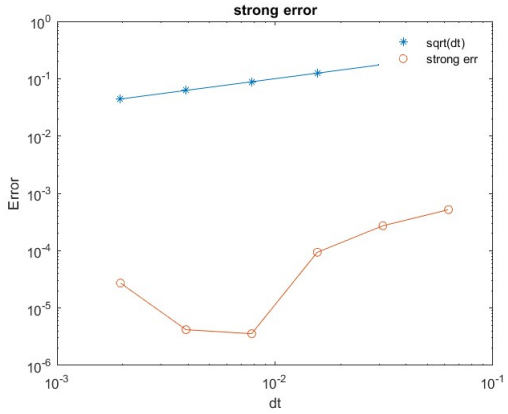


(a) strong error without exact sol

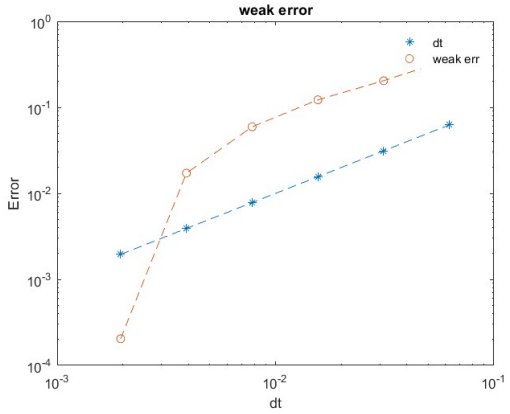


(b) weak error without exact sol

Figure 3: g(x)=x



(a) strong error without exact sol



(b) weak error without exact sol

Figure 4: g(x)=e^x

(There is a difference in the two graphs of strong error, even if it should not be affected by the change of g, because every time the code is run, the vector containing the increments of Wiener process is changed (it is random)).
What is more important to see, is that even with this different implementation, the weak error seems to behave in a better way according to theory when g is chosen such that it satisfies theorem 5.8.

2 Exercise 2

2.1

Considering the ordinary differential equation

$$dX_t = AX_t dt$$

with $X_t \in \mathbf{R^2}$ and the matrix A has two real eigenvalues $\lambda_1 = 1$ and $\lambda_2 = -10^5$, we want to show why that the backward Euler method

$$X(t_{n+1}) - X(t_n) = AX(t_{n+1})(t_{n+1} - t_n)$$

is an efficient method to solve the problem.
We can start rearranging the Backward Euler equation:

$$(I - A\Delta t)X(t_{n+1}) = X(t_n)$$

$$\Rightarrow X(t_{n+1}) = (I - A\Delta t)^{-1}X(t_n)$$

with $\Delta t = t_{n+1} - t_n$.
Making a change of variables, we can show that the Backward Euler scheme is efficient in solving this problem since it admits any kind of time discretization, while other methods like Forward Euler, in order to mantain stability of the method, need to put some constraints to the lenght of the time-steps, depending on the eigenvalues of the system's matrix.
Let $A = PDP^{-1}$, where $D = diag(\lambda_1, \lambda_2)$ and let $X = P^{-1}X$, then

$$dY_t = DY_t dt$$

with a certain initial condition

$$Y(0) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

the analytical solution is given by

$$Y(t) = e^{Dt}Y(0) = \begin{bmatrix} e^t & 0 \\ 0 & e^{-10^5 t} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} y_1 e^t \\ y_2 e^{-10^5 t} \end{bmatrix}$$

Now changing variables in the Backward Euler scheme:

$$(I - D\Delta t_n)Y_{n+1} = Y_n \Rightarrow Y_{n+1} = \begin{bmatrix} \frac{1}{1-\Delta t_n} & 0 \\ 0 & \frac{1}{1+10^5 \Delta t_n} \end{bmatrix} Y_n$$

which implies:

$$\Rightarrow Y_n = \begin{bmatrix} \frac{1}{1-\Delta t_n} & 0 \\ 0 & \frac{1}{1+10^5 \Delta t_n} \end{bmatrix}^n Y_0$$

Now, considering for example a uniform time discretization ($\Delta t_n = T/N \quad \forall n$), the error of the Backward Euler method is:

$$E_{BW} = \begin{bmatrix} |(e^T - (\frac{1}{1-\frac{T}{N}})^N)|y_1 \\ |(e^{-10^5 T} - (\frac{1}{1+10^5 \frac{T}{N}})^N)|y_2 \end{bmatrix}$$

In order to show the benefits of Backward Euler, we compare this error with Forward Euler's error, that we can get following the same procedure:

$$E_{FW} = \begin{bmatrix} |(e^T - (1 + \frac{T}{N})^N)|y_1 \\ |(e^{-10^5 T} - (1 - 10^5 \frac{T}{N})^N)|y_2 \end{bmatrix}$$

Plugging in the values $T = 1$, $y_1 = 1$, $y_2 = 1$ and $N = 100$ we can already see a huge difference between the two methods:

$$E_{BW} = \begin{bmatrix} 0.0137 \\ \approx 0 \end{bmatrix}$$

$$E_{FW} = \begin{bmatrix} 0.0135 \\ 9.048 * 10^{299} \end{bmatrix}$$

while backward Euler's error converges to 0 as N increases, the error of forward Euler relative to the second component explodes, this is because forward Euler's stability is guaranteed only for time steps satisfying the following inequalities:

$$\Delta t \leq \frac{2}{|\lambda_{max}|}$$

that in this case is

$$\Delta t \leq \frac{2}{10^5} = 0.00002 \Rightarrow \frac{T}{N} \leq 0.00002 \Rightarrow N \geq \frac{10^5}{2}$$

2.2

Let's formulate the backward Euler method for the approximation of the Ito SDE:

$$dX_t = aX_t dt + bX_t dW_t$$

as

$$X(t_{n+1}) - X(t_n) = aX(t_{n+1})(t_{n+1} - t_n) + bX(t_{n+1})(W(t_{n+1}) - W(t_n))$$

or, simplifying the notation, as:

$$X_{n+1} - X_n = aX_{n+1}\Delta t_n + bX_{n+1}\Delta W_n$$

then, rearranging:

$$X_{n+1} = \frac{X_n}{1 - a\Delta t_n - b\Delta W_n}$$

At this point we want to modify the drift term in order to make this approximation more consistent with the Ito solution of the SDE. Thus, let's look at the exact solution of the SDE:

$$X_t = X(0)e^{(a-\frac{b^2}{2})t+bW_t}$$

the derivation of the solution follows these steps:

- use Ito's lemma on the transformation
$$Y_t = ln(X_t)$$
obtaining the linear SDE
$$dY_t = (a - \frac{b^2}{2})dt + bdW_t$$
- integrate both sides of the linear equation in order to find Y_t:
$$Y_t = Y(0) + (a - \frac{b^2}{2})t + bW_t$$
with $Y(0)$ a certain initial condition
- finally obtaining the solution X_t through the exponentiation of $Y_t = ln(X_t)$:
$$X_t = X(0)e^{(a-\frac{b^2}{2})t+bW_t}$$
assuming $X(0) = e^{Y(0)}$

We see that the coefficient of the drift term in the Ito solution is $(a - \frac{b^2}{2})$, while in the backward Euler approximation was only a. In order to improve consistency, we can modify the drift term, making the approximation:

$$X_{n+1} = \frac{X_n}{1 - (a - \frac{b^2}{2})\Delta t_n - b\Delta W_n}$$

Even after this modification, the backward Euler scheme applied to SDE can still have a problem theoretically. The problem is about the stability of the method, in fact it follows that:

$$X_n = X(0)\prod_{i=1}^{n-1}\frac{1}{1 - (a - \frac{b^2}{2})\Delta t_i - b\Delta W_i}$$

and in order to have stability, we want the coefficients

$$\frac{1}{1 - (a - \frac{b^2}{2})\Delta t_i - b\Delta W_i}$$

to be bounded (in particular when the time discretization is uniform, we want this coefficient to be ≤ 1, so that X_n does not explode for $n \to \infty$). The drift part of the denominator is always greater than 0 (thanks to the hypothesis that $a < 0$) but the diffusion term represents a problem, since $\Delta W_i \sim \mathcal{N}(0, \Delta t_i)$.
and could lead to a division bt 0. So, in practice, we should use time fine time discretization in order to keep ΔW_i as bounded as possible reducing its variance, meaning that the backward Euler scheme is not a good method to approximate stochastic differential equations over a coarse time discretization.

2.3

In order to formulate an implicit method that avoids the problem described in the previous section, we can change the modified backward Euler in the following way:

$$X_{n+1} - X_n = (a - \frac{b^2}{2})X_{n+1}\Delta t_n + bX_n\Delta W_n$$

rearranging:

$$X_{n+1} = X_n \frac{1 + b\Delta W_n}{1 - (a - \frac{b^2}{2})\Delta t}$$

The method is still implicit and inherits the consistency with the Ito solution given by the modified drift term, while the diffusion term is treated explicitly, avoiding the stability issue of the possible division by 0 found in section 2.b